Mathematica ile Karmaşık Formlar

Üretken sanat için ileri teknikler rehberi — kaos_ve_form.wl projesini geliştiren Claude Code oturumları için hazırlandı. Her bölümde fikir, neden işe yaradığı ve doğrudan kullanılabilir Wolfram Language kodu var.

0. Claude Code için çalışma kuralları

- Mathematica bu makinede çalıştırılamayabilir; kodu yazdıktan sonra parantez/köşeli parantez dengesini, virgülleri ve `Function` argümanlarını satır satır kontrol et.
- Mevcut dosya .wl paket formatında: bölümler `(* ::Section:: *)`, açıklamalar `(* ::Text:: *)` ile başlar. Bu yapıyı koru, mevcut tanımları (attractorArt, gradients, art1–art5) silme.
- Türkçe karakterler dosyada `\:0131`, `\[CCedilla]` gibi kaçışlarla yazılmış; yeni metinde de aynı biçimi ya da düz UTF-8 kullan, ikisini aynı satırda karıştırma.
- Ağır işlemleri (PlotPoints > 80, milyonlarca nokta, NDSolve) önce düşük ayarla önizlenecek şekilde yaz; yüksek kalite değerlerini ayrı bir değişkende tut.
- Sürüme bağlı fonksiyonlarda (MaterialShading 13+, ComplexPlot 12+) `$VersionNumber` kontrolü ekle.

1. Örtük yüzeyleri karmaşıklaştırmak

Gyroid gibi bir yüzey f(x,y,z)=0 denklemidir. Karmaşıklık, f'yi değiştirerek gelir: koordinatları bükmek, birden fazla yüzeyi harmanlamak, farklı ölçekleri üst üste koymak.

1a. Alan bükme (domain warping)

Fonksiyona girmeden önce koordinatları sinüslerle kaydır. Düzenli kafes organik, akışkan bir forma dönüşür. a büyüdükçe bozulma artar; iki kez uygulamak (iç içe warp) mermer benzeri dokular verir.

```
gyroid[{x_, y_, z_}] := Sin[x] Cos[y] + Sin[y] Cos[z] + Sin[z] Cos[x];
warp[{x_, y_, z_}, a_] := {x + a Sin[2 y], y + a Sin[2 z], z + a Sin[2 x]};

ContourPlot3D[gyroid[warp[warp[{x, y, z}, 0.5], 0.3]] == 0,
  {x, -Pi, Pi}, {y, -Pi, Pi}, {z, -Pi, Pi},
  RegionFunction -> Function[{x, y, z}, x^2 + y^2 + z^2 < Pi^2],
  Mesh -> None, PlotPoints -> 70, MaxRecursion -> 1,
  Boxed -> False, Axes -> False, Background -> Black]
```

1b. Kalınlaştırılmış kabuk ve çok ölçekli detay

`Abs[f] == t` sonsuz ince yüzeyi t kalınlığında katı bir kabuğa çevirir (3B baskıya da uygun). Aynı fonksiyonu 3 kat sıklıkta ve küçük genlikle eklemek ikinci bir detay katmanı yaratır.

```
f[p_] := gyroid[p] + 0.25 gyroid[3 p] + 0.08 gyroid[9 p];
ContourPlot3D[Abs[f[{x, y, z}]] == 0.18, {x, -Pi, Pi}, {y, -Pi, Pi}, {z, -Pi, Pi},
  Mesh -> None, PlotPoints -> 90, MaxRecursion -> 0]
```

1c. Yumuşak birleşim (smooth union)

İki yüzeyi keskin kesişim yerine eriyerek birleşmiş gibi göstermek için yumuşak minimum kullanılır. k küçüldükçe geçiş yumuşar.

```
schwarzP[{x_, y_, z_}] := Cos[x] + Cos[y] + Cos[z];
smin[a_, b_, k_] := -Log[Exp[-k a] + Exp[-k b]]/k;

g[p_] := smin[Abs[gyroid[p]] - 0.15, Abs[schwarzP[2 p]] - 0.2, 6];
ContourPlot3D[g[{x, y, z}] == 0, {x, -Pi, Pi}, {y, -Pi, Pi}, {z, -Pi, Pi},
  Mesh -> None, PlotPoints -> 80]
```

1d. Diğer üçlü periyodik minimal yüzeyler

```
diamond[{x_, y_, z_}] := Sin[x] Sin[y] Sin[z] + Sin[x] Cos[y] Cos[z] +
                         Cos[x] Sin[y] Cos[z] + Cos[x] Cos[y] Sin[z];
neovius[{x_, y_, z_}] := 3 (Cos[x] + Cos[y] + Cos[z]) + 4 Cos[x] Cos[y] Cos[z];
lidinoid[{x_, y_, z_}] := 0.5 (Sin[2x] Cos[y] Sin[z] + Sin[2y] Cos[z] Sin[x] +
     Sin[2z] Cos[x] Sin[y]) - 0.5 (Cos[2x] Cos[2y] + Cos[2y] Cos[2z] + Cos[2z] Cos[2x]) + 0.15;
```

2. Parametrik yüzeyler: Süperformül

Gielis süperformülü, birkaç parametreyle denizyıldızından çiçeğe, kabuktan kristale uzanan formlar üretir. İki süperformülün küresel çarpımı 3B bir yüzey verir. `m` simetri sayısıdır.

```
sf[t_, m_, n1_, n2_, n3_] := (Abs[Cos[m t/4]]^n2 + Abs[Sin[m t/4]]^n3)^(-1/n1);

superShape[{m1_, a1_, b1_, c1_}, {m2_, a2_, b2_, c2_}] :=
 ParametricPlot3D[
  With[{r1 = sf[u, m1, a1, b1, c1], r2 = sf[v, m2, a2, b2, c2]},
   {r1 Cos[u] r2 Cos[v], r1 Sin[u] r2 Cos[v], r2 Sin[v]}],
  {u, -Pi, Pi}, {v, -Pi/2, Pi/2},
  PlotPoints -> 150, Mesh -> None, Boxed -> False, Axes -> False,
  ColorFunction -> Function[{x, y, z, u, v}, ColorData["SunsetColors"][Rescale[v, {-Pi/2, Pi/2}]]],
  ColorFunctionScaling -> False, Background -> Black];

superShape[{7, 0.2, 1.7, 1.7}, {7, 0.2, 1.7, 1.7}]
superShape[{5, 0.1, 1.7, 1.7}, {1, 0.3, 0.5, 0.5}]
```

3. 3B kaos: tüp şeklinde çekiciler

2B Clifford çekicisinin 3B karşılığı: diferansiyel denklemlerle tanımlanan bir yörünge. NDSolve ile çözüp yörüngeyi hıza göre renklendirilmiş çizgi ya da Tube olarak çizmek ipliksi, heykelsi bir form verir.

```
(* Thomas çekicisi — b ~ 0.208 kaotik bölge *)
b = 0.208;
sol = NDSolveValue[{
    x'[t] == Sin[y[t]] - b x[t],
    y'[t] == Sin[z[t]] - b y[t],
    z'[t] == Sin[x[t]] - b z[t],
    x[0] == 0.1, y[0] == 0., z[0] == 0.}, {x, y, z}, {t, 0, 1500},
   MaxSteps -> Infinity];

pts = Table[Through[sol[t]], {t, 100, 1500, 0.04}];     (* ilk geçişi at *)
speed = Rescale[Norm /@ Differences[pts]];
Graphics3D[{AbsoluteThickness[0.6],
   Line[Most[pts], VertexColors -> (ColorData["SunsetColors"] /@ speed)]},
 Background -> Black, Boxed -> False, ImageSize -> 800,
 Lighting -> "Neutral"]
```

Diğer denemeler: Lorenz (σ=10, ρ=28, β=8/3), Aizawa, Halvorsen, Rössler. Kalın görünüm için `Tube[BSplineCurve[pts[[;; ;; 10]]], 0.02]` kullan (nokta sayısını azaltarak).

4. Işık ve malzeme

Aynı geometri, ışıkla bambaşka görünür. Varsayılan ışığı kapatıp sıcak/soğuk iki renkli nokta ışık kullanmak sinematik bir derinlik verir.

```
lights = {{"Ambient", GrayLevel[0.08]},
          {"Point", RGBColor[1, 0.55, 0.25], {6, 4, 5}},
          {"Point", RGBColor[0.25, 0.45, 1], {-6, -3, 2}},
          {"Directional", GrayLevel[0.35], ImageScaled[{0, 0, 2}]}};

(* herhangi bir 3B grafiğe: *)
Show[plot, Lighting -> lights]

(* yarı saydam, parlak cam görünüm *)
PlotStyle -> Directive[Opacity[0.55], Specularity[White, 50], Glow[RGBColor[0.05, 0, 0.1]]]
```

```
(* Mathematica 13+ : fiziksel malzemeler *)
If[$VersionNumber >= 13, PlotStyle -> MaterialShading["Gold"]]
```

5. 2B: daha zengin yoğunluk haritaları

5a. Parlama (bloom) efekti

Görüntünün bulanık bir kopyasını kendisine eklemek, parlak bölgelerin ışık saçıyormuş gibi görünmesini sağlar.

```
bloom[g_, r_ : 25, s_ : 0.9] := Module[{img = Rasterize[g, ImageResolution -> 200]},
   ImageAdd[img, ImageMultiply[GaussianFilter[img, r], s]]];
bloom[art1]
```

5b. Üst üste katmanlar

Farklı parametrelerle üretilmiş iki çekiciyi ekleyerek (Screen benzeri) birleştir; renk paletlerini farklı seç.

```
layer1 = Rasterize[attractorArt[{-1.4, 1.6, 1.0, 0.7}, 3000000, 1000, "Ateş"], ImageResolution -> 150];
layer2 = Rasterize[attractorArt[{-1.7, 1.3, -0.1, -1.2}, 3000000, 1000, "Okyanus"], ImageResolution -> 150];
ImageAdd[layer1, ImageMultiply[ImageResize[layer2, ImageDimensions[layer1]], 0.7]]
```

5c. Newton fraktalı (Compile ile)

```
newton = Compile[{{c, _Complex}},
   Module[{z = c, i = 0},
    While[i < 40 && Abs[z^5 - 1] > 10.^-6, z = z - (z^5 - 1)/(5 z^4); i++];
    {Arg[z], N[i]}], RuntimeAttributes -> {Listable}, RuntimeOptions -> "Speed"];

grid = Table[x + I y, {y, -1.5, 1.5, 0.003}, {x, -1.5, 1.5, 0.003}];
data = newton[grid];
Image[Map[List @@ ColorConvert[
   Blend[{Black, Hue[(#[[1]] + Pi)/(2 Pi)]}, 1 - #[[2]]/40.], "RGB"] &, data, {2}]]
```

6. 3B fraktal nokta bulutları

Kaos oyunu 3B'de: rastgele bir köşeye doğru yarı yol gidip her adımda küçük bir dönme eklemek, Sierpinski tetrahedronunu bükülmüş, spiralli bir yapıya dönüştürür.

```
chaos3D = Compile[{{n, _Integer}, {twist, _Real}},
   Module[{p = {0., 0., 0.}, v, c = Cos[twist], s = Sin[twist],
     verts = {{1., 1., 1.}, {1., -1., -1.}, {-1., 1., -1.}, {-1., -1., 1.}}},
    Table[v = verts[[RandomInteger[{1, 4}]]];
     p = 0.5 (p + v);
     p = {c p[[1]] - s p[[2]], s p[[1]] + c p[[2]], p[[3]]};
     p, {n}]], RuntimeOptions -> "Speed"];

pts = chaos3D[400000, 0.35];
Graphics3D[{PointSize[0.0015],
   Point[pts, VertexColors -> (ColorData["DeepSeaColors"] /@ Rescale[pts[[All, 3]]])]},
 Background -> Black, Boxed -> False, ImageSize -> 800]
```

7. Fiziksel çıktı: 3B baskı ve video

```
(* Kalınlaştırılmış gyroid kabuğunu STL olarak dışa aktar *)
shell = ContourPlot3D[Abs[gyroid[{x, y, z}]] == 0.3, {x, -2Pi, 2Pi}, {y, -2Pi, 2Pi}, {z, -2Pi, 2Pi},
   Mesh -> None, PlotPoints -> 60];
Export[FileNameJoin[{$HomeDirectory, "Desktop", "gyroid_shell.stl"}], DiscretizeGraphics[shell]];

(* Kare listesinden video *)
frames = Table[Show[plot, ViewPoint -> 3 {Cos[t], Sin[t], 0.5}], {t, 0, 2Pi - 2Pi/120, 2Pi/120}];
Export[FileNameJoin[{$HomeDirectory, "Desktop", "donus.mp4"}], frames, "FrameRate" -> 30];
```

8. Performans ipuçları

Sorun	Çözüm
ContourPlot3D çok yavaş	PlotPoints 40–60 ile önizle, MaxRecursion -> 0 veya 1
Milyonlarca iterasyon	Compile + RuntimeOptions -> "Speed" (gerekirse CompilationTarget -> "C")
Animasyon kareleri	ParallelTable (önce LaunchKernels[])
Manipulate takılıyor	ControlActive[düşük, yüksek] ile sürüklerken düşük kalite
Bellek doluyor	Ara sonuçları Rasterize ile görüntüye çevir; $HistoryLength = 0

9. Resmi kaynaklar

- ContourPlot3D — https://reference.wolfram.com/language/ref/ContourPlot3D.html
- ParametricPlot3D — https://reference.wolfram.com/language/ref/ParametricPlot3D.html
- Graphics3D — https://reference.wolfram.com/language/ref/Graphics3D.html
- Lighting — https://reference.wolfram.com/language/ref/Lighting.html
- MaterialShading — https://reference.wolfram.com/language/ref/MaterialShading.html
- Compile — https://reference.wolfram.com/language/ref/Compile.html
- NDSolveValue — https://reference.wolfram.com/language/ref/NDSolveValue.html
- ComplexPlot — https://reference.wolfram.com/language/ref/ComplexPlot.html
- Manipulate — https://reference.wolfram.com/language/ref/Manipulate.html
- Export (video) — https://reference.wolfram.com/language/ref/format/MP4.html
- Wolfram Demonstrations Project — https://demonstrations.wolfram.com
- Wolfram Community — https://community.wolfram.com